

# StudySync: Building Resilient Distributed Systems

Sara Noor | BSCS23162

## Part 1: Analyze the Mess

---

StudySync's MVP is a distributed full-stack system: React clients, a FastAPI backend, a database, Clerk, and an external LLM API all communicate over unreliable boundaries. The current architecture fails because it assumes requests arrive once, in order, quickly, and without concurrency. Those assumptions are false in real distributed systems.

### Problem 1: Synchronization / Lost Update

The synchronization bug occurs across the read-modify-write lifecycle. User A and User B both read document version v1 from the API/database. Each client edits locally, then sends a PUT request containing only the new content. The backend writes the incoming content without checking whether the row changed after the client read it. In the database this becomes a last-write-wins update: whichever transaction commits last overwrites the earlier commit. The API returns success to both users, so one change disappears silently. The root cause is missing concurrency control: no version column, compare-and-swap condition, row lock, merge algorithm, or conflict response.

### Problem 2: Coordination / Lost Clerk Webhook

Clerk coordinates subscription state by sending webhook events such as `subscription.cancelled` to the backend. In the naive design, the backend treats the webhook as a single chance to update local user state. If the network blips, the backend is down, or the request times out before durable processing, the cancellation is never recorded. Clerk and StudySync now disagree: Clerk says cancelled, while the local database still says premium. The root cause is non-durable, non-idempotent event handling with no retry acknowledgement strategy, event log, reconciliation job, or dead-letter path.

### Problem 3: Fault Tolerance / Blocking LLM Call

The external LLM API is a remote dependency with independent latency and failures. If FastAPI waits synchronously for a 60-second LLM timeout inside a request handler, that dependency becomes a single point of failure. With limited workers or blocking I/O, slow LLM calls consume server capacity and unrelated users can experience timeouts. The root cause is coupling user-facing request availability directly to a slow external service without timeouts, bulkheads, async execution, circuit breaking, or fallback behavior.

## Part 2: Design a Better System

---

### Sync Fix: Optimistic Locking

Add a version integer to each document. A client must send the version it last read when saving. The backend performs an atomic conditional update: update document content only if `current_version == client_version`, then increment the version. If another user already updated the document, the condition fails and the API returns 409 Conflict with the latest document. The stale client can then reload, merge, and retry. This preserves consistency without pessimistically locking the document during user editing time.

### UML Sequence Diagram

#### Coordination Fix: Durable Webhook Processing

The Clerk webhook endpoint should verify the signature, store the event in a `webhook_events` table using Clerk's event ID as an idempotency key, and acknowledge only after the event is durably stored. A background worker processes unhandled events and updates subscription state inside a transaction. Duplicate deliveries are safe because the event ID is unique. Failed processing attempts are retried with exponential backoff. After the retry limit, the event moves to a dead-letter queue for alerting and manual repair. A scheduled reconciliation job can also compare local premium users against Clerk's API to catch rare missed states.

#### Fault Tolerance Fix: Circuit Breaker with Fallback

Wrap LLM calls in a circuit breaker. In the closed state, calls are allowed but use a short timeout. After repeated failures or timeouts, the breaker opens and immediately returns a fallback such as "AI summary temporarily unavailable; try again later" or a cached previous response. After a cooldown, the breaker enters half-open and allows one trial request. If it succeeds, the breaker closes; if it fails, it opens again. This prevents one failing dependency from exhausting FastAPI capacity.

#### CAP and Trade-offs

For shared document edits, StudySync chooses stronger consistency over blind availability: a stale write receives 409 Conflict instead of being accepted. That slightly increases user-visible latency because clients may need to

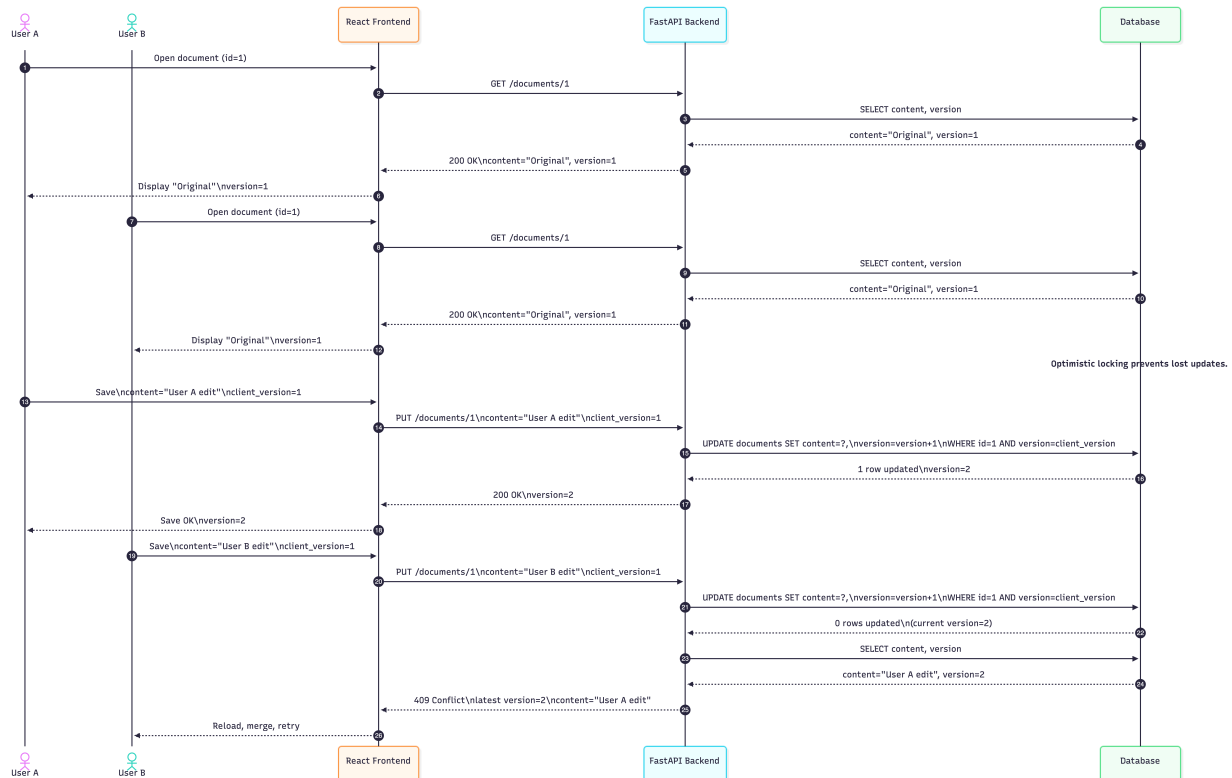


Figure 1: Description of my screenshot

refresh and merge, but it prevents silent data loss. For webhooks, the system favors eventual consistency and durability: cancellation may be processed asynchronously, but the event is stored and retried until handled. For LLM features, the system favors availability and low latency over perfect freshness: fallback responses keep the core app usable even when the LLM is unavailable.

### Part 3: Code Implemented

This repo implements optimistic locking for the document synchronization problem. The fixed endpoint is `PUT /docs/{doc_id}` and requires a version in the request body. If two users edit version 1, the first update succeeds and increments the document to version 2; the second update receives HTTP 409 Conflict. The broken behavior is still available at `PUT /naive/docs/{doc_id}` for demo comparison. Tests in `test_sync.py` prove the lost update in the naive endpoint, prove the conflict behavior in the fixed endpoint, and verify that every response contains X-Student-ID: BSCS23162.